Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
Second Semester 2024-2025

Comprehensive Examination
(EC-3 Regular)

Course No.	: AIMLCZG511
Course Title	: Deep Neural Networks
Nature of Exam	: Open Book
Weightage	: 40%
Duration	: 2 Hours
Date of Exam	: 14-09-2025

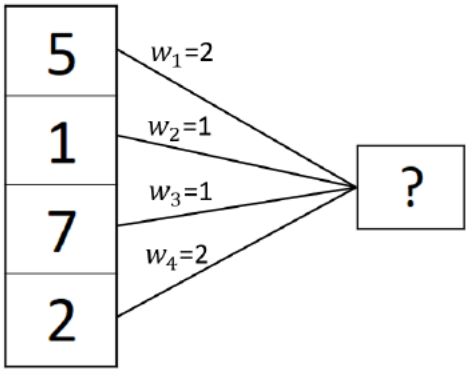
No. of Pages = 03
No. of Questions = 06

Note to Students:
1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. Assumptions made if any, should be stated clearly at the beginning of your answer.

Q.1. Answer the following questions: (limit answers to 1-2 sentences) **[5 Marks]**

A. Gradients that are close to zero can be a problem when training a network. Suggest four different techniques that can reduce the problem.

B. Given the dense layer shown below. What would the maximum value of the output be, assuming a dropout probability of p=0.5?



C. Given the dense layer shown above. What would the minimum value of the output be, assuming a dropout probability of p=0.5?

Note: For B and C, provide two interpretation cases.

Answer Key:

A. **Four ways to reduce near-zero gradients.**
Use ReLU-family activations (ReLU/Leaky-ReLU), use proper initialization (He/ or Xavier), add normalization layers (BatchNorm/LayerNorm), and add residual/skip connections.

B. The two cases:
1. Only 50% of the nodes will be active, so finding the maximum is equivalent to finding the two nodes that result in the maximum output.
Maximum value: (5*2 + 0 *1 + 7 * 1 + 0 * 2)/0.5 = 34
2. Over time, with a dropout of 0.5, all nodes will be on, so the maximum value can be: Maximum value: (5*2 + 1*1 + 7 *1 + 2 *2)/0.5 = 44

C. The two cases:

1. Only 50% of the nodes will be active, so finding the minimum is equivalent to finding the two nodes that result in the minimum output.
Minimum value: (0*2 + 1*1 + 0*1 + 2*2)/0.5 = 10
2. Over time, with dropout 0.5, all nodes will be dropped, so the minimum value can be 0.

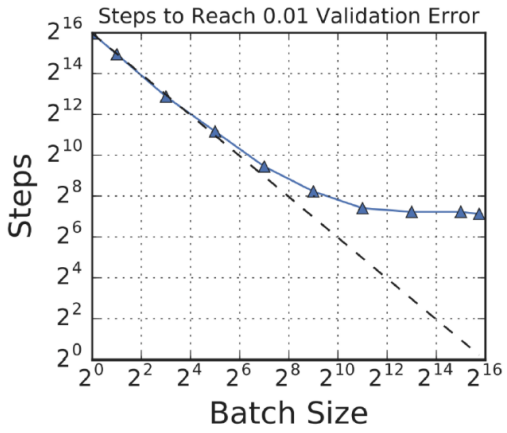
Q.2. Answer the following (limit answers to 1-2 sentences) **[5 Marks]**

A. Suppose you set up and train a neural network on a classification task and converge to a final loss value. Keeping everything in the training process the exact same (e.g. learning rate, optimizer, epochs). It is possible to reach a lower loss value by ONLY changing the network initialization. (1M)

B. What is a commonly used loss function when training a neural network for a multi-class classification problem? (1M)

C. What is the key reason why backpropagation is so important? (1M)

D. The following plot of the number of SGD iterations required to reach a given loss, as a function of the batch size:



- For small batch sizes, the number of iterations required to reach the target loss decreases as the batch size increases. Why is that? (1M)
- For large batch sizes, the number of iterations does not change much as the batch size is increased. Why is that? (1M)

Answer Key:

A. Yes, different weight initializations can lead to different optimization paths and potentially lower final loss, even with the same training setup.

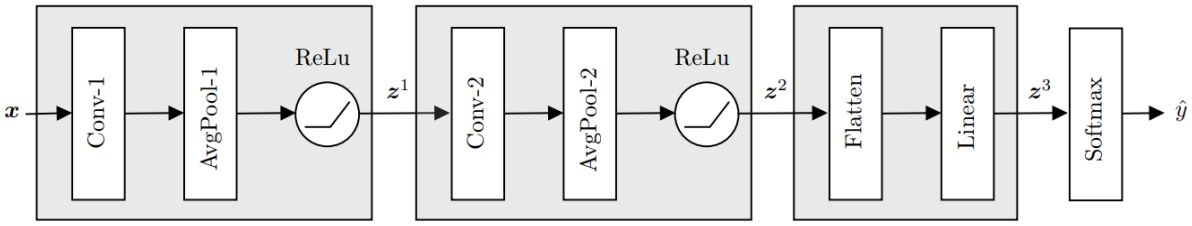
B. Cross-Entropy Loss is the most commonly used for multi-class classification.

C. Backpropagation efficiently computes gradients for all parameters by applying the chain rule, making large-scale neural network training feasible.

D (i). For small batches, a larger batch size reduces gradient noise and gives a more accurate estimate of the true gradient, hence fewer iterations.

(ii). For large batches, gradients are already accurate, so increasing batch size further gives diminishing returns in computation increases, but convergence rate doesn't improve much.

Q.3. You are given a CNN trained on a dataset of **32×32 grayscale images**, where each image x has a corresponding class label y∈{0,1,…,9}. The CNN architecture is as follows: **[10 Marks]**



- Conv-1: 8 filters, kernel size 5×5, stride 1, padding 0
- AvgPool-1: kernel size 4×4, stride 4, padding 0
- Conv-2: 16 filters, kernel size 4×4, stride 1, padding 1
- AvgPool-2: kernel size 2×2, stride 2, padding 0
- Flatten → Linear → Softmax

Answer the following questions.

A. What is the key advantage of using pooling layers in a neural network? What differences would you expect between using a max pooling layer in comparison to an average pooling layer? (1M)

B. Compute the output dimensions (height × width × channels) after each layer, starting from the input size 32×32×1. Show all calculations clearly. (Assume bias = 1 per filter). (4M)

C. A student implemented CNN in PyTorch as shown in the snapshot below. The code produces a dimension mismatch error during the linear layer. Identify the error and correct the layer configuration so that the code runs properly. Write the corrected PyTorch block. Justify your answer. (3M)

```
nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=8, kernel_size=5, stride=1, padding=0),
    nn.AvgPool2d(kernel_size=4, stride=4, padding=0)
    nn.ReLU(),
    nn.Conv2d(in_channels=8, out_channels=16, kernel_size=4, stride=1, padding=1),
    nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
    nn.ReLU(),
    nn.Flatten(),
    nn.Linear(120, 10)
)
```

D. Instead of using Flatten + Linear, it is possible to use a convolutional layer to project directly into $\mathbb{R}^{10}$. Describe the convolutional layer configuration (kernel size, number of filters, and expected output size) that can replace the final block while still allowing classification via a Softmax layer. (2M)

Answer Key:

A. Pooling layers provide invariance to the neural network, making the model more robust to small variations in the input, and increasing the receptive field of neurons in the deeper layers of the network. Max pooling layers select the maximum value for each patch of the feature map, while average pooling layers calculate the average of all values within the patch. While max pooling layers select the most prominent feature for each patch of the feature map, average pooling layers smoothen out all the feature values within the patch

B. Layer Conv-1: num. weights = num. of filters × ((kernel width × kernel height × num. channels) + bias) = 8 × ((5 × 5 × 1) + 1).
Layer Conv-2 : num. weights = num. of filters × ((kernel width × kernel height × num. channels) + bias) = 16 × ((4 × 4 × 8) + 1).
Layers AvgPool-1 and AvgPool-2 do not have trainable weights.

C. The input dimensions are 32 × 32 × 1

The output of layer *Conv-1*, is of size $28 \times 28 \times 8$, where 8 is the number of channels of the convolutional layer and $\frac{\text{input dim.}-\text{kernel size}}{\text{stride}} + 1 = \frac{32-5}{1} + 1 = 28$.

The output of layer *AvgPool-1*, is of size $7 \times 7 \times 8$, where 8 is the number of channels (the same number of input channels of the layer) and $\frac{\text{input dim.}-\text{kernel size}}{\text{stride}} + 1 = \frac{28-4}{4} + 1 = 7$.

The output of layer *Conv-2*, is of size $6 \times 6 \times 16$, where 16 is due to the number of kernels. First, we pad the input of the layer yielding size $9 \times 9 \times 8$. Second, this input is convoluted with the kernels yielding an output dimension of $\frac{\text{input dim.}-\text{kernel size}}{\text{stride}} + 1 = \frac{9-4}{1} + 1 = 6$.

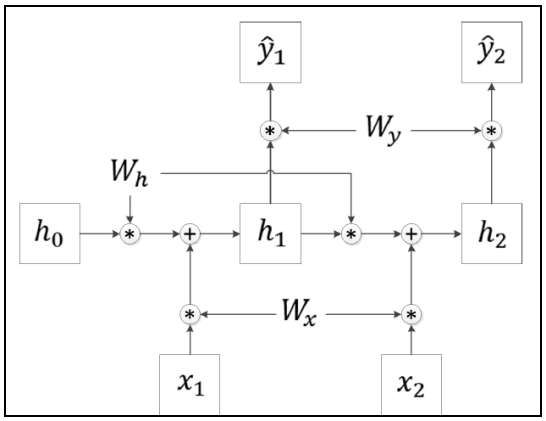
The output of layer *AvgPool-2*, is of size $3 \times 3 \times 16$, where 16 is the number of channels (the same number of input channels of the layer) and $\frac{\text{input dim.}-\text{kernel size}}{\text{stride}} + 1 = \frac{6-2}{2} + 1 = 3$.

Thus, after the flattening operation we obtain a vector of dimension $3 \times 3 \times 16 = 144$. Hence, the linear layer of the network should be initialized as `nn.Linear(144, 10)`.

D. Solution v1: The output of the second pooling layer AvgPool-2 is 3×3×16, hence we know that the input dimension of the convolution layer would be 16. Thus, if we apply a kernel with size 3 × 3 and with 10 output channels (filters) we will obtain outputs of dimension 1 × 1 × 10.
Solution v2: The output of the second pooling layer AvgPool-2 is 3 × 3 × 16, hence we know that the input dimension of the convolution layer would be 16. If we apply a kernel with size 1 × 1 and with 10 output channels (filters) we will obtain outputs of dimension 3 × 3 × 10. If we then apply a third pooling layer AvgPool-3 of size 3 × 3 we will obtain outputs of dimension 1 × 1 × 10.

Q.4. Answer the following questions **[10 Marks]**

A. You are given a simple RNN network as illustrated in the computational graph below. Assume that we use identity functions as activation functions. Assume that the input at a given time, the hidden state, and the output at a given time are scalar. Let $h_0 = 1$, $x_1 = 10$, $x_2 = 10$, $y_1 = 5$, $y_2 = 5$. Assume the initial values of the weights are: $W_h = 1$, $W_x = 0.1$, $W_y = 2$.



- Compute the predicted value y_2
- If we use quadratic loss, the loss at a given time t is,

$$L_t = (y_t - \hat{y}_t)^2,$$

Compute the total loss given the values for the weights and inputs given above. (4M)

B. A company is forecasting weekly sales using three different recurrent models: Vanilla RNN, LSTM, and GRU. At week t, the input x_t=10, the previous hidden state h_{t-1}=2. The weight values are initialized as follows (assume scalar case, and all activations are identity functions unless specified): (6M)

Vanilla RNN: $h_t = W_h h_{t-1} + W_x x_t$, with $W_h = 0.5,\ W_x = 0.1$.
LSTM: forget gate $f_t = \sigma(0)$, input gate $i_t = \sigma(1)$, candidate $\tilde{c}_t = \tanh(1)$, output gate $o_t = \sigma(1)$. Previous cell state $c_{t-1} = 2$.
GRU: reset gate $r_t = \sigma(0)$, update gate $z_t = \sigma(1)$, candidate $\tilde{h}_t = \tanh(W_h(r_t \cdot h_{t-1}) + W_x x_t)$ with $W_h = 0.5, W_x = 0.1$.

- Compute the hidden state h_t for the Vanilla RNN
- Compute the new cell state c_t and hidden state h_t for the LSTM.
- Compute the hidden state h_t for the GRU.
- Given the weekly sales forecasting scenario and the computed states above, which model (Vanilla RNN, LSTM, or GRU) is most appropriate to deploy and why?

.

Answer Key:

A.

$$h_1 = W_x x_1 + W_h h_0 = (0.1 * 10 + 1 * 1) = 2$$

$$\hat{y}_2 = W_y(W_h h_1 + W_x x_2) = 2(1 * 2 + 0.1 * 10) = 6$$

$$\hat{y}_1 = W_y(W_h h_0 + W_x x_1) = 2(1 * 1 + 0.1 * 10) = 4$$

$$L_1 = (\hat{y}_1 - y_1)^2 = (4 - 5)^2 = 1$$

$$L_2 = (\hat{y}_2 - y_2)^2 = (6 - 5)^2 = 1$$

$$L = L_1 + L_2 = 2$$

B.

1) Vanilla RNN:

$$h_t = W_h h_{t-1} + W_x x_t = 0.5 \cdot 2 + 0.1 \cdot 10 = 1 + 1 = 2$$

2) LSTM:

$$c_t = f_t c_{t-1} + i_t c_t = 0.5 \cdot 2 + 0.7311 \cdot 0.7616 \approx 1 + 0.5568 = 1.5568$$
$$h_t = o_t \tanh(c_t) \approx 0.7311 \cdot \tanh(1.5568) \approx 0.7311 \cdot 0.9149 \approx 0.6688$$

3) GRU:

$$h_t = \tanh(W_h(r_t h_{t-1}) + W_x x_t) = \tanh(0.5 \cdot (0.5 \cdot 2) + 0.1 \cdot 10) = \tanh(1.5) \approx 0.9051$$
$$h_t = (1 - z_t) h_{t-1} + z_t h_t \approx (1 - 0.7311) \cdot 2 + 0.7311 \cdot 0.9051 \approx 0.2689 \cdot 2 + 0.662 \approx 1.1996$$

4) GRU is generally the best trade-off here: it captures temporal dependencies with gating (mitigating vanishing gradients) like LSTM, but with fewer parameters and often faster convergence, well-suited for weekly sales where medium/long-term patterns matter but model simplicity and data efficiency are important. (If very long seasonality and abundant data are emphasized, LSTM is also reasonable; vanilla RNN is least appropriate due to stability/gradient issues.)

Q.5. Assume you are developing a Transformer-based system for analyzing event-based vision data. Suppose at a certain timestep, the sensor produces a pixel feature embedding for a detected object: **[5 Marks]**

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

The model uses a single-head self-attention mechanism with the following learnable weight matrices:

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

A. Compute the query (Q), key (K), and value (V) vectors for the input. (2M)
B. Compute the attention score and the softmax attention weight. (2M)
C. Compute the final self-attention output. (1M)

Answer Key:

A.

$$Q = W_Q x = Ix = \boxed{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}$$

$$K = W_K x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$V = W_V x = \begin{bmatrix} 1 \cdot 1 + 3 \cdot 2 \\ 2 \cdot 1 + 4 \cdot 2 \end{bmatrix} = \boxed{\begin{bmatrix} 7 \\ 10 \end{bmatrix}}$$

B.

Scaled dot-product score (with $d_k = 2$):

$$s = \frac{Q^\top K}{\sqrt{d_k}} = \frac{[1,2] \cdot [2,1]}{\sqrt{2}} = \frac{4}{\sqrt{2}} = \boxed{2.828 \text{ (approx.)}}.$$

With a single token, $\text{softmax}(s) = \boxed{1.0}$.

C.

$$\text{Output} = \alpha V = 1 \cdot \boxed{\begin{bmatrix} 7 \\ 10 \end{bmatrix}}.$$

Q.6. Answer the following questions: **[5 Marks]**

I) Consider a time series $u_1, u_2, \ldots, u_t, \ldots$, with $u_t \in \mathbb{R}^3$ and $t \in N$, which verifies (3M)

$$u_{t+1} = 5\max(2u_t - 3u_{t-1} - u_{t-2} + 1, 0), \text{ and}$$

Assume $u_t = 0$ for $t < 0$. Suppose that we want to build a predictor for the time series using a neural network. In particular, we want to build a neural network that takes as input $x = [u^T_{t-2}, u^T_{t-1}, u^T_t]^T$ and returns as output $y = u_{t+1}$. Show that the dynamics of the time series can be represented exactly using a neural network with a single hidden layer. In particular, indicate the dimension, weights, and activation function of both hidden and output layers

II) You are tasked with designing a lightweight CNN for deployment on a mobile device with limited computation. Instead of manually tuning layers, you decide to use NAS. Which component of NAS will determine how different candidate architectures are explored, and why? (2M)

Answer Key:

A.

$$W_1 = \begin{bmatrix} -1 & 0 & 0 & -3 & 0 & 0 & 2 & 0 & 0 \\ 0 & -1 & 0 & 0 & -3 & 0 & 0 & 2 & 0 \\ 0 & 0 & -1 & 0 & 0 & -3 & 0 & 0 & 2 \end{bmatrix}, \qquad b_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

The output of the hidden layer is thus a vector $\boldsymbol{h} \in \mathbb{R}^3$ with

$$\begin{aligned} \boldsymbol{h}(\boldsymbol{x}) &= \mathrm{ReLU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) \\ &= \mathrm{ReLU}\left(\begin{bmatrix} 2u_{t,1} - 3u_{t-1,1} - u_{t-2,1} + 1 \\ 2u_{t,2} - 3u_{t-1,2} - u_{t-2,2} + 1 \\ 2u_{t,3} - 3u_{t-1,3} - u_{t-2,3} + 1 \end{bmatrix}\right). \end{aligned}$$

The output layer consists of 3 linear units, and

$$\boldsymbol{W}_o = 5\,\mathbf{I}_{3\times 3}, \qquad \boldsymbol{b}_o = \mathbf{0}.$$

The output thus comes $\boldsymbol{y} = 5\,\boldsymbol{h}(\boldsymbol{x})$.

B. The search strategy/controller (e.g., RL controller, evolutionary algorithm, Bayesian optimization, or random search) determines how candidate architectures are explored because it samples and updates choices from the search space, balancing exploration vs. exploitation using validation rewards/constraints (e.g., accuracy–latency).